ABM Project 보안/리팩토링/정리 감사 보고서
작성일: 2026-06-27 (Asia/Seoul)
대상 경로: /Users/seunghyuk-kim/Desktop/ABM_project
작업 방식: 읽기 전용 정적 점검. 기존 소스 파일 수정/삭제 없음.

요약
- 우선 처리 권장: JWT 쿠키 CSRF 방어, DM 권한 검증, 프로필 학적 검증 우회, 파일 업로드 검증,
  민감 이미지 저장소 공개 여부, 관리자 OTP 보호.
- 중간 우선순위: 공개 API throttle, 댓글 parent 정합성, PII 암호화 설계, 로그 입력 길이 검증,
  프론트 보안 헤더, 외부 AI 전송 데이터 정책.
- 정리/리팩토링: community/networks 중복, 대형 page/views 파일, 빈/미사용 파일, 혼재 라우트,
  테스트 placeholder 정리.

1. 보안 관련 주요 허점

- [HIGH] JWT 쿠키 기반 인증에서 CSRF 방어가 약함
    영향: access/refresh token을 HttpOnly 쿠키로 사용하고 SameSite=None 구성이 존재합니다. DR
인증 경로는 CSRF 검사를
      강제하지 않는 경우가 많아, 상태 변경 API가 크로스사이트 요청에 노출될 수 있습니다.
    근거: backend/config/settings.py:345; backend/config/settings.py:426;
      backend/apps/users/authentication.py:4
    권장: 쿠키 JWT를 유지한다면 상태 변경 API에 CSRF 토큰 검증을 강제하거나, Authorization 헤더 기
SameSite/Origin
      검증 정책을 명확히 하세요.

- [HIGH] 토큰이 JSON 응답과 localStorage에도 저장됨
    영향: HttpOnly 쿠키를 쓰면서 access/refresh를 JSON으로도 내려주고 프론트에서 localStorage에
 XSS 발생 시 토큰 탈취
      가능성이 커집니다.
    근거: backend/apps/users/views.py:647; backend/apps/users/views.py:748;
      backend/apps/users/views.py:855; frontend/src/shared/api/api.ts:12
    권장: 운영에서는 HttpOnly 쿠키 또는 Authorization 헤더 중 하나로 일관화하고, localStorage 토
거하는 방향이 안전합니다.

- [HIGH] Direct Message 생성 권한 검증 누락
    영향: MessageSerializer의 conversation이 writable이고 perform_create에서 요청 사용자가
대화 participant인지
      확인하지 않습니다. 대화 ID를 알면 타인의 대화에 메시지를 삽입할 수 있습니다.
    근거: backend/apps/direct_messages/serializers.py:8; backend/apps/direct_messa
ges/views.py:56
    권장: perform_create에서 conversation.participants에 request.user가 포함되는지 검증하고
rializer에서
      conversation 선택 범위를 사용자 소유 대화로 제한하세요.

- [HIGH] Direct Message 수정/삭제 범위 과다
    영향: MessageViewSet queryset은 conversation 참여자 기준이라, 같은 대화 참여자가 상대방 메시
삭제할 수 있는 구조일
      가능성이 있습니다.
    근거: backend/apps/direct_messages/views.py:49
    권장: 수정/삭제는 sender 본인 또는 관리자만 가능하도록 object permission을 분리하세요.

- [HIGH] 프로필 수정으로 온보딩 학적 검증 우회 가능
    영향: 온보딩에서는 학번 형식, 중복, StudentRegistry, 이름 일치, 입학연도 등을 검증하지만 프로필 수
r는
      student_id/name/grade/department/admission_year를 거의 그대로 받습니다.
    근거: backend/apps/users/views.py:123; backend/apps/users/serializers.py:56;
      backend/apps/users/serializers.py:251; frontend/src/app/(main)/profile/edi
t/page.tsx:152
    권장: 검증 완료 후 바뀌면 안 되는 필드는 read-only 처리하거나, 변경 시 온보딩과 동일한 검증/재승인

- [HIGH] network draft 이미지 업로드 검증 및 소유권 연결 부족

    영향: ImageUploadView가 업로드 파일을 거의 검증하지 않고 post=None인 PostFile로 저장합니다. _id를 통해 임의 파일
      연결이 가능해질 수 있습니다.
    근거: backend/apps/networks/views.py:600; backend/apps/networks/views.py:610;
      backend/apps/networks/serializers.py:366
    권장: 업로드 파일 MIME/확장자/크기 검증, 업로더 소유자 필드, draft 전용 모델 또는 임시 토큰을 도입하

- [HIGH] community 첨부파일 검증 함수가 실제 입력명과 어긋남
    영향: serializer 필드는 new_files인데 validate_files가 정의되어 있어 DRF 자동 검증이 호출되지 update 경로도
      PostDetailSerializer를 사용해 sanitize/update 로직이 죽어 있을 수 있습니다.
    근거: backend/apps/community/serializers.py:217; backend/apps/community/serializers.py:242;
      backend/apps/community/views.py:110
    권장: validate_new_files로 맞추고, create/update 모두 동일한 sanitizer와 파일 검증 경로를 사용하세요.

- [HIGH] 관리자 OTP 검증에 시도 제한/잠금이 부족하고 난수도 random 기반
    영향: API OTP verify가 AllowAny이고 실패 횟수 제한이 보이지 않습니다. OTP 생성도 random.randint를 사용합니다.
    근거: backend/apps/users/views.py:440; backend/apps/users/views.py:800;
      backend/apps/admin_otp/views.py:25
    권장: secrets 기반 난수, user_id/IP별 시도 제한, 실패 횟수 잠금, OTP 해시 저장을 적용하세요.

- [HIGH] 복수전공 증빙 이미지가 public R2 설정과 충돌할 가능성
    영향: 접근 endpoint는 권한을 확인하지만, R2 설정은 public bucket 시 서명 URL 불필요(querystring_auth=False)라는
      전제입니다. 실제 버킷이 public이면 민감 이미지가 우회 접근될 수 있습니다.
    근거: backend/config/settings.py:190; backend/config/settings.py:206;
      backend/apps/users/models.py:175; backend/apps/users/views.py:866
    권장: 민감 증빙 이미지는 별도 private bucket/storage에 저장하고, 짧은 만료 signed URL만 허용하세요. R2 정책을 확인하세요.

- [MEDIUM] 댓글 parent가 같은 게시글 소속인지 검증하지 않음
    영향: 게시글 A에 댓글을 달면서 parent는 게시글 B의 댓글 ID를 넣을 수 있어 댓글 트리, 알림, 카운트가
    근거: backend/apps/community/views.py:221; backend/apps/community/serializers.py:356;
      backend/apps/networks/views.py:266; backend/apps/networks/serializers.py:579
    권장: parent가 존재하면 parent.post_id == 현재 post.id인지 serializer 또는 view에서 강제

- [MEDIUM] 학번/닉네임 존재 여부 공개 API가 enumeration에 취약
    영향: check_nickname, check_student_id가 AllowAny이고 별도 throttle이 보이지 않습니다. 학번은 대량 조회로 가입/존재
      여부 추정이 가능합니다.
    근거: backend/apps/users/views.py:235; backend/apps/users/views.py:253
    권장: IP/세션별 throttle, captcha 또는 generic response를 검토하세요.

- [MEDIUM] PageViewLog 공개 write API에 throttle이 없음
    영향: AllowAny page-view endpoint가 공개 쓰기이며 별도 제한이 보이지 않습니다. 로그 테이블 부풀림에 악용될 수 있습니다.
    근거: backend/logs/views.py:827; backend/logs/urls.py:56
    권장: 익명 로그 API용 throttle, payload 크기 제한, bot 필터링을 적용하세요.

- [MEDIUM] PII 암호화 효과가 제한적
    영향: email/student_id 원문 필드가 계속 남아 있고 hash도 salt 없는 SHA-256입니다. 학번처럼 좁은 값은 사전 공격에
      약합니다.
    근거: backend/apps/users/models.py:77; backend/apps/users/models.py:112;
      backend/apps/users/utils_encryption.py:25
    권장: 검색용 해시는 keyed HMAC/pepper를 쓰고, 원문 필드 유지 필요성을 재검토하세요.

- [MEDIUM] EventLog 일부 문자열 길이 검증 누락
    영향: 모델 max_length가 있는 page, utm_source 등이 저장 전 잘리지 않아 PostgreSQL DataEr
00을 유발할 수 있습니다.
    근거: backend/logs/models.py:24; backend/logs/utils.py:153; backend/logs/views
.py:833
    권장: 로그 저장 전 모든 max_length 필드에 공통 truncation/validation을 적용하세요.

- [MEDIUM] 자기 글/댓글 좋아요 및 reaction race 처리 미흡
    영향: 본인 콘텐츠 좋아요 제한이 없고 reaction 생성에 transaction/IntegrityError 처리가 부족하
 500, like_count
      음수화가 가능합니다.
    근거: backend/apps/users/utils_score.py:38; backend/apps/community/views.py:17
3;
      backend/apps/community/views.py:513; backend/apps/networks/views.py:218
    권장: 자기 콘텐츠 반응 제한 정책, DB unique 충돌 처리, F expression 후 음수 clamp를 적용하세요

- [MEDIUM] Next 프론트 보안 헤더/CSP 부재
    영향: next.config.ts에는 image 설정만 있고 보안 헤더가 보이지 않습니다. GTM inline script와
torage 토큰 구조까지
      고려하면 XSS 방어층이 약합니다.
    근거: frontend/next.config.ts:1; frontend/src/app/layout.tsx:36;
      backend/config/settings.py:390
    권장: Next headers()로 CSP, X-Content-Type-Options, Referrer-Policy 등 프론트 응답 헤
를 추가하세요.

- [MEDIUM] 관리자 인사이트가 검색어를 외부 Anthropic API로 전송
    영향: 검색 키워드가 개인정보를 포함할 수 있는데, 관리자 인사이트 생성 시 외부 AI 서비스로 전달됩니다.
    근거: backend/logs/views.py:1619
    권장: 전송 데이터 최소화, 개인정보 마스킹, 관리자 UI 고지, 개인정보 처리방침 반영 여부를 확인하세요.

2. 코드 리팩토링 필요 부분

- [REFACTOR] community와 networks 도메인 코드 중복
    영향: Post, Comment, Reaction, 파일, 썸네일, 좋아요, 알림 흐름이 거의 유사합니다. 같은 버그가
능성이 큽니다.
    근거: backend/apps/community/serializers.py; backend/apps/networks/serializers
.py;
      backend/apps/community/views.py; backend/apps/networks/views.py
    권장: 공통 mixin/service를 먼저 작은 단위로 추출하고, 게시판별 차이만 설정값으로 분리하세요.

- [REFACTOR] 대형 파일 분리 필요
    영향: 관리자 페이지와 로그 view, 네트워크 작성/목록 페이지가 매우 큽니다. 변경 영향 추적과 테스트가 어
    근거: backend/logs/views.py:1; frontend/src/app/admin/page.tsx:1;
      frontend/src/app/network/page.tsx:1; frontend/src/app/network/write_net/pa
ge.tsx:1
    권장: API/service/view/component 단위로 나누고, 관리자 통계 query helper를 별도 모듈로 분리

- [REFACTOR] TypeScript any 사용이 넓음
    영향: API 응답과 form 상태에서 any가 넓게 퍼져 있으면 런타임 필드 누락을 컴파일 단계에서 잡기 어렵습
    근거: frontend/src
    권장: 공유 API response 타입과 form DTO를 먼저 정의하고, 자주 쓰는 페이지부터 any를 줄이세요.

- [REFACTOR] logs/views.py가 분석, 관리자 API, AI 인사이트, 유저 관리까지 담당
    영향: 하나의 views 파일에 역할이 과도하게 집중되어 있습니다. 성능 최적화와 권한 검토가 어려워집니다.
    근거: backend/logs/views.py:1; backend/logs/views.py:1252; backend/logs/views.
py:1619
    권장: analytics, admin_user_management, ai_insights, event_ingest 등으로 관심사를 분리
세요.

- [REFACTOR] 라우트/페이지 명명 혼재
    영향: network/write와 network/write_net처럼 기능이 겹쳐 보이는 경로가 섞여 있습니다.

    근거: frontend/src/app/network/write_net/page.tsx:1
    권장: 실제 사용 경로를 기준으로 하나의 canonical route를 정하고 legacy route는 redirect/삭제
류하세요.

- [REFACTOR] 운영성능 리스크: unbounded list와 GET side effect
    영향: 관리자 유저 목록 무페이지네이션, 대화 메시지 전체 로딩, 알림 목록 조회 시 오래된 알림 삭제 같은
    근거: backend/logs/views.py:1252; backend/apps/direct_messages/views.py:25;
      backend/apps/notifications/views.py:23
    권장: pagination, cursor, background cleanup job, 명시적 mutation endpoint를 적용하세
.

3. 불필요하거나 정리할 코드/경로/API 후보

- [CLEANUP] 루트의 파일명 3
    영향: 의미를 알 수 없는 루트 파일입니다. 산출물/실수 생성 파일일 가능성이 있습니다.
    근거: 3
    권장: 내용과 생성 이력을 확인한 뒤 삭제 여부를 결정하세요.

- [CLEANUP] 루트 package-lock.json이 사실상 빈 lockfile
    영향: 루트 package.json 없이 package-lock.json만 있으면 혼란을 줍니다.
    근거: package-lock.json
    권장: 프론트 lockfile과 역할이 중복되는지 확인하고, 필요 없다면 제거 후보로 분류하세요.

- [DOCS] frontend README가 기본 템플릿 수준
    영향: 프로젝트 실행, 환경변수, 배포, 테스트 방법이 실제 프로젝트 기준으로 정리되어 있지 않습니다.
    근거: frontend/README.md
    권장: 개발자 온보딩 문서로 갱신하세요.

- [CLEANUP] 빈 admin_otp/models.py
    영향: Django app 관례상 남아 있을 수 있지만 실제 모델이 없다면 유지 필요성이 낮습니다.
    근거: backend/apps/admin_otp/models.py
    권장: 관례 유지인지 불필요 파일인지 팀 기준으로 정하세요.

- [CLEANUP] 미사용 가능성이 높은 post service 파일
    영향: features/post/services.ts가 실제 import되지 않는 것으로 보입니다.
    근거: frontend/src/features/post/services.ts
    권장: 사용처 확인 후 삭제 또는 실제 API service로 통합하세요.

- [CLEANUP] mobile/network의 빈 오타 파일들
    영향: MobileCoummunity* 파일들이 0바이트이고 import도 발견되지 않았습니다. Coummunity 오타도
    근거: frontend/src/features/mobile/network/MobileCoummunityDetailPage.tsx;
      frontend/src/features/mobile/network/MobileCoummunityListPage.tsx;
      frontend/src/features/mobile/network/MobileCoummunityWritePage.tsx
    권장: 사용 계획이 없다면 제거 후보, 필요하다면 Community 명칭으로 재생성하세요.

- [CLEANUP] placeholder 테스트/미완성 문자열
    영향: 의미 있는 검증 없이 placeholder로 남은 테스트나 화면의 생략 문자열은 유지보수 신뢰도를 떨어뜨
    근거: backend/apps/*/tests.py; frontend/src/app/network/*
    권장: 실제 regression 테스트로 바꾸거나 제거 후보로 분류하세요.

- [CLEANUP] public top_active_users의 하드코딩 제외 목록
    영향: 특정 닉네임 제외가 코드와 monthly reset command에 중복 하드코딩되어 있습니다.
    근거: backend/apps/users/views.py:759;
      backend/apps/users/management/commands/reset_monthly_scores.py:19
    권장: User 모델 또는 별도 정책 테이블에 exclude_from_rankings 같은 플래그로 관리하세요.

- [CONFIG] 입학연도 상한 2025 하드코딩
    영향: 2026년 기준 신규 입학연도 처리에 문제가 생길 수 있습니다.
    근거: backend/apps/users/serializers.py:317; frontend/src/app/(auth)/onboardin
g/page.tsx:456;
      frontend/src/app/(main)/profile/edit/page.tsx:428

권장: 현재연도 또는 설정값 기반으로 바꾸고 백엔드/프론트 상한을 공유하세요.

4. 권장 처리 순서

1) 인증/권한: CSRF, localStorage token, DM conversation 권한, 프로필 학적 검증 우회.
2) 파일/민감정보: community/network 업로드 검증, R2 private media 정책, PII hash/encryptio
설계.
3) 남용 방지: OTP attempt lock, 공개 enumeration/page-view throttle, EventLog 길이 검증.
4) 데이터 정합성: 댓글 parent 소속 검증, self-like/race 처리, like_count clamp.
5) 운영성: pagination, GET side effect 제거, 관리자/로그 대형 view 분리.
6) 정리: 빈/미사용 파일, 혼재 route, placeholder 테스트/문서 정비.

5. 이번 PDF 작성 시 검증 제약

- npm audit는 외부 npm audit 서비스로 의존성 정보를 전송해야 해서 승인 정책상 실행되지 않았습니다.
- pip-audit는 로컬에 설치되어 있지 않았습니다.
- backend/venv와 frontend/node_modules가 없어 Django check, Next build/test 같은 실행 검
은 하지 못했습니다.
- 본 보고서는 로컬 파일 읽기 기반의 정적 점검 결과입니다.
- 기존 소스 파일은 수정하거나 삭제하지 않았고, 새 PDF 산출물만 생성했습니다.